FFT Programs

As discussed in Chapter 8, the *real DFT* can be calculated by correlating the time domain signal with sine and cosine waves (see Table 8-2). Table 12-2 shows a program to calculate the *complex DFT* by the same method. In an apples-to-apples comparison, this is the program that the FFT improves upon.

Tables 12-3 and 12-4 show two different FFT programs, one in FORTRAN and one in BASIC. First we will look at the BASIC routine in Table 12-4. This subroutine produces exactly the same output as the correlation technique in Table 12-2, except it does it *much faster*. The block diagram in Fig. 12-7 can be used to identify the different sections of this program. Data are passed to this FFT subroutine in the arrays: REX[ ] and IMX[ ], each running from sample 0 to $N-1$. Upon return from the subroutine, REX[ ] and IMX[ ] are overwritten with the frequency domain data. This is another way that the FFT is highly optimized; the same arrays are used for the input, intermediate storage, and output. This efficient use of memory is important for designing fast hardware to calculate the FFT. The term **in-place computation** is used to describe this memory usage.

While all FFT programs produce the same numerical result, there are subtle variations in programming that you need to look out for. Several of these

```
5000 'COMPLEX DFT BY CORRELATION
5010 'Upon entry, N% contains the number of points in the DFT, and
5020 'XR[ ] and XI[ ] contain the real and imaginary parts of the time domain.
5030 'Upon return, REX[ ] and IMX[ ] contain the frequency domain data.
5040 'All signals run from 0 to N%-1.
5050 '
5060 PI = 3.14159265                    'Set constants
5070 '
5080 FOR K% = 0 TO N%-1                 'Zero REX[ ] and IMX[ ], so they can be used
5090   REX[K%] = 0                      'as accumulators during the correlation
5100   IMX[K%] = 0
5110 NEXT K%
5120 '
5130 FOR K% = 0 TO N%-1                 'Loop for each value in frequency domain
5140   FOR I% = 0 TO N%-1               'Correlate with the complex sinusoid, SR & SI
5150 '
5160     SR =  COS(2*PI*K%*I%/N%)       'Calculate complex sinusoid
5170     SI  = -SIN(2*PI*K%*I%/N%)
5180     REX[K%] = REX[K%] + XR[I%]*SR - XI[I%]*SI
5190     IMX[K%] = IMX[K%] + XR[I%]*SI + XI[I%]*SR
5200 '
5210   NEXT I%
5220 NEXT K%
5230 '
5240 RETURN
```

TABLE 12-2